

ACTIVITY 1: Understanding the Bare Bones Account Class (Python)

account.py

```
"""
A simple Account class for the Global Digital Bank system.
This is a bare bones implementation with minimal functionality.
"""

class Account:
    """
    A simple bank account class with basic deposit and withdraw functionality.
    """

    def __init__(self, account_number, name, age, initial_balance, account_type):
        """
        Creates a new bank account.

        Args:
            account_number (int): Unique account number
            name (str): Account holder's name
            age (int): Account holder's age
            initial_balance (float): Initial deposit amount
            account_type (str): "Savings" or "Current"
        """
        self.account_number = account_number
        self.name = name
        self.age = age
        self.balance = initial_balance
        self.account_type = account_type
        self.status = "Active" # New accounts are active by default

    def deposit(self, amount):
        """
        Deposits money into the account.
        """
```

```

    Args:
        amount (float): Amount to deposit
    """
    if amount > 0:
        self.balance = self.balance + amount
        print(f"Deposited: ₹{amount}")
        print(f"New balance: ₹{self.balance}")
    else:
        print("Invalid amount. Deposit failed.")

def withdraw(self, amount):
    """
    Withdraws money from the account.

    Args:
        amount (float): Amount to withdraw
    """
    if amount > 0 and amount <= self.balance:
        self.balance = self.balance - amount
        print(f"Withdrawn: ₹{amount}")
        print(f"New balance: ₹{self.balance}")
    elif amount > self.balance:
        print("Insufficient balance. Withdrawal failed.")
    else:
        print("Invalid amount. Withdrawal failed.")

def show_balance(self):
    """
    Displays the account balance.
    """
    print(f"Account #{self.account_number}")
    print(f"Balance: ₹{self.balance}")

def get_account_info(self):
    """
    Returns account details as a formatted string.

    Returns:
        str: Formatted account information
    """
    return (f"Account #{self.account_number} | {self.name} "
            f"({self.age} yrs) | {self.account_type} | ")

```

```

        f"₹{self.balance} | {self.status}")

    def __str__(self):
        """String representation of the account."""
        return self.get_account_info()

```

test_account.py

```

"""
Simple test driver for the Account class.
"""

from account import Account

def main():
    print("=" * 50)
    print("  GLOBAL DIGITAL BANK – ACCOUNT TEST")
    print("=" * 50)
    print()

    # 1. Create an account
    print(">>> 1. Creating Account")
    acc1 = Account(1001, "John Doe", 25, 1000.0, "Savings")
    print("Account created!")
    print(acc1.get_account_info())
    print()

    # 2. Deposit money
    print(">>> 2. Deposit Money")
    acc1.deposit(500.0)
    print()

    # 3. Withdraw money
    print(">>> 3. Withdraw Money")
    acc1.withdraw(200.0)
    print()

    # 4. Try to withdraw more than balance
    print(">>> 4. Withdraw More Than Balance")
    acc1.withdraw(2000.0)
    print()

```

```

# 5. Check balance
print(">>> 5. Check Balance")
acc1.show_balance()
print()

# 6. Create another account
print(">>> 6. Creating Another Account")
acc2 = Account(1002, "Jane Smith", 30, 2000.0, "Current")
print(acc2.get_account_info())
print()

# 7. Display all accounts
print(">>> 7. All Accounts")
print(acc1.get_account_info())
print(acc2.get_account_info())
print()

print("=" * 50)
print("  TEST COMPLETED!")
print("=" * 50)

if __name__ == "__main__":
    main()

```

Expected Output

```

=====
GLOBAL DIGITAL BANK – ACCOUNT TEST
=====

>>> 1. Creating Account
Account created!
Account #1001 | John Doe (25 yrs) | Savings | ₹1000.0 | Active

>>> 2. Deposit Money
Deposited: ₹500.0
New balance: ₹1500.0

>>> 3. Withdraw Money
Withdrawn: ₹200.0

```

New balance: ₹1300.0

```
>>> 4. Withdraw More Than Balance
Insufficient balance. Withdrawal failed.
```

```
>>> 5. Check Balance
Account #1001
Balance: ₹1300.0
```

```
>>> 6. Creating Another Account
Account #1002 | Jane Smith (30 yrs) | Current | ₹2000.0 | Active
```

```
>>> 7. All Accounts
Account #1001 | John Doe (25 yrs) | Savings | ₹1300.0 | Active
Account #1002 | Jane Smith (30 yrs) | Current | ₹2000.0 | Active
```

```
=====
TEST COMPLETED!
=====
```

ACTIVITY 2: Enhancing the Account Class (Python)

account_enhanced.py

```
"""
Enhanced Account class with validation, minimum balance, status management
, and PIN protection.
"""

class Account:
    """
    Enhanced bank account class with validation and security features.
    """

    # Class constants for minimum balances
    MIN_BALANCE_SAVINGS = 500.0
    MIN_BALANCE_CURRENT = 1000.0
```

```
MIN_AGE = 18
```

```
def __init__(self, account_number, name, age, initial_balance, account_type):
```

```
    """
```

```
    Creates a new bank account with validation.
```

```
    Args:
```

```
        account_number (int): Unique account number
```

```
        name (str): Account holder's name
```

```
        age (int): Account holder's age (must be >= 18)
```

```
        initial_balance (float): Initial deposit amount
```

```
        account_type (str): "Savings" or "Current"
```

```
    """
```

```
    self.account_number = account_number
```

```
    self.name = name
```

```
    self.age = age
```

```
    self.balance = initial_balance
```

```
    self.account_type = account_type
```

```
    self.status = "Active" # New accounts are active by default
```

```
    self.pin = None # PIN not set by default
```

```
    # Validate age
```

```
    self._validate_age()
```

```
    # Validate account type
```

```
    self._validate_account_type()
```

```
    # Validate minimum balance
```

```
    self._validate_minimum_balance()
```

```
def _validate_age(self):
```

```
    """Validates that account holder is at least 18 years old."""
```

```
    if self.age < self.MIN_AGE:
```

```
        print(f"Error: Customer must be at least {self.MIN_AGE} years  
old")
```

```
        print(f"Setting age to {self.MIN_AGE}")
```

```
        self.age = self.MIN_AGE
```

```
def _validate_account_type(self):
```

```
    """Validates that account type is either 'Savings' or 'Current'."""
```

```
    "
```

```

        if self.account_type not in ["Savings", "Current"]:
            print("Error: Account type must be 'Savings' or 'Current'")
            print("Defaulting to 'Savings'")
            self.account_type = "Savings"

    def _validate_minimum_balance(self):
        """Validates that initial balance meets minimum requirements."""
        min_balance = self._get_minimum_balance()
        if self.balance < min_balance:
            print(f"Error: {self.account_type} account needs minimum ₹{min_
balance}")
            print(f"Setting balance to minimum ₹{min_balance}")
            self.balance = min_balance

    def _get_minimum_balance(self):
        """Returns the minimum balance for the account type."""
        if self.account_type == "Savings":
            return self.MIN_BALANCE_SAVINGS
        else: # Current
            return self.MIN_BALANCE_CURRENT

    def _is_active(self):
        """Checks if the account is active."""
        return self.status == "Active"

    def _verify_pin(self, pin):
        """Verifies the account PIN."""
        if self.pin is None:
            print("Error: PIN not set for this account")
            return False
        return self.pin == pin

    def deposit(self, amount):
        """
        Deposits money into the account.

        Args:
            amount (float): Amount to deposit
        """
        # Check if account is active
        if not self._is_active():
            print("Error: Account is inactive. Cannot deposit.")

```

```

        return

    if amount > 0:
        self.balance = self.balance + amount
        print(f"Deposited: ₹{amount}")
        print(f"New balance: ₹{self.balance}")
    else:
        print("Invalid amount. Deposit failed.")

def withdraw(self, amount, pin=None):
    """
    Withdraws money from the account.

    Args:
        amount (float): Amount to withdraw
        pin (int, optional): 4-digit PIN for verification
    """
    # Check if account is active
    if not self._is_active():
        print("Error: Account is inactive. Cannot withdraw.")
        return

    # Verify PIN if set
    if self.pin is not None:
        if pin is None:
            print("Error: PIN required for withdrawal")
            return
        if not self._verify_pin(pin):
            print("Error: Incorrect PIN.")
            return

    # Check minimum balance
    min_balance = self._get_minimum_balance()

    if amount > 0 and (self.balance - amount) >= min_balance:
        self.balance = self.balance - amount
        print(f"Withdrawn: ₹{amount}")
        print(f"New balance: ₹{self.balance}")
    elif amount > self.balance:
        print("Insufficient balance. Withdrawal failed.")
    elif (self.balance - amount) < min_balance:

```



```

        print(f"Error: Cannot withdraw. Minimum balance of ₹{min_balance} required.")
    else:
        print("Invalid amount. Withdrawal failed.")

def show_balance(self):
    """Displays the account balance."""
    print(f"Account #{self.account_number}")
    print(f"Balance: ₹{self.balance}")

def close_account(self):
    """Closes the account by marking it as inactive."""
    if not self._is_active():
        print(f"Account #{self.account_number} is already closed.")
        return
    self.status = "Inactive"
    print(f"Account #{self.account_number} has been closed.")

def reopen_account(self):
    """Reopens a closed account by marking it as active."""
    if self._is_active():
        print(f"Account #{self.account_number} is already active.")
        return
    self.status = "Active"
    print(f"Account #{self.account_number} has been reopened.")

def set_pin(self, pin):
    """
    Sets a 4-digit PIN for the account.

    Args:
        pin (int): 4-digit number (1000-9999)
    """
    if 1000 <= pin <= 9999:
        self.pin = pin
        print("PIN set successfully.")
    else:
        print("Error: PIN must be a 4-digit number (1000-9999).")

def verify_pin(self, pin):
    """
    Verifies if the provided PIN matches the account PIN.

```

```

    Args:
        pin (int): PIN to verify

    Returns:
        bool: True if PIN matches, False otherwise
    """
    if self.pin is None:
        print("Error: PIN not set for this account")
        return False
    return self.pin == pin

def get_account_info(self):
    """
    Returns account details as a formatted string.

    Returns:
        str: Formatted account information
    """
    pin_status = "PIN Set" if self.pin is not None else "No PIN"
    return (f"Account #{self.account_number} | {self.name} "
            f"({self.age} yrs) | {self.account_type} | "
            f"₹{self.balance} | {self.status} | {pin_status}")

def __str__(self):
    """String representation of the account."""
    return self.get_account_info()

```

test_account_enhanced.py

```

"""
Test driver for the enhanced Account class.
"""

from account_enhanced import Account

def main():
    print("=" * 60)
    print("  ENHANCED ACCOUNT TEST")
    print("=" * 60)
    print()

```

```

# Test 1: Valid account creation
print(">>> Test 1: Valid Account Creation")
acc1 = Account(1001, "John Doe", 25, 1000.0, "Savings")
print(acc1.get_account_info())
print()

# Test 2: Invalid age
print(">>> Test 2: Invalid Age (under 18)")
acc2 = Account(1002, "Young Kid", 16, 500.0, "Savings")
print(acc2.get_account_info())
print()

# Test 3: Invalid account type
print(">>> Test 3: Invalid Account Type")
acc3 = Account(1003, "Test User", 25, 1000.0, "Invalid")
print(acc3.get_account_info())
print()

# Test 4: Minimum balance enforcement
print(">>> Test 4: Minimum Balance Enforcement")
print("Creating Savings account with ₹300 (below minimum)")
acc4 = Account(1004, "Bob Wilson", 25, 300.0, "Savings")
print(acc4.get_account_info())
print()

# Test 5: Withdrawal with minimum balance
print(">>> Test 5: Withdrawal with Minimum Balance")
acc5 = Account(1005, "Alice Brown", 30, 1000.0, "Current")
print(f"Initial: {acc5.get_account_info()}")
acc5.withdraw(200.0)
print(f"After withdrawing ₹200: {acc5.get_account_info()}")
print("Trying to withdraw ₹900 (would leave ₹-100)")
acc5.withdraw(900.0)
print()

# Test 6: Account status
print(">>> Test 6: Account Status Management")
acc6 = Account(1006, "Charlie Green", 35, 2000.0, "Savings")
print(f"Initial: {acc6.get_account_info()}")
acc6.close_account()
print(f"After close: {acc6.get_account_info()}")

```

```

print("Trying to deposit to closed account:")
acc6.deposit(500.0)
acc6.reopen_account()
print(f"After reopen: {acc6.get_account_info()}")
print()

# Test 7: PIN protection
print(">>> Test 7: PIN Protection")
acc7 = Account(1007, "Diana Prince", 28, 1500.0, "Savings")
acc7.set_pin(1234)
print("Withdraw with correct PIN (1234):")
acc7.withdraw(200.0, 1234)
print("Withdraw with incorrect PIN (9999):")
acc7.withdraw(100.0, 9999)
print("Withdraw without providing PIN:")
acc7.withdraw(100.0)
print()

# Test 8: Display all accounts
print(">>> Test 8: All Accounts Summary")
accounts = [acc1, acc2, acc3, acc4, acc5, acc6, acc7]
for acc in accounts:
    print(acc.get_account_info())
print()

print("=" * 60)
print("  ENHANCED TEST COMPLETED!")
print("=" * 60)

if __name__ == "__main__":
    main()

```

Expected Output

```

=====
ENHANCED ACCOUNT TEST
=====

```

```

>>> Test 1: Valid Account Creation
Account #1001 | John Doe (25 yrs) | Savings | ₹1000.0 | Active | No PIN

```

>>> Test 2: Invalid Age (under 18)

Error: Customer must be at least 18 years old

Setting age to 18

Account #1002 | Young Kid (18 yrs) | Savings | ₹500.0 | Active | No PIN

>>> Test 3: Invalid Account Type

Error: Account type must be 'Savings' or 'Current'

Defaulting to 'Savings'

Account #1003 | Test User (25 yrs) | Savings | ₹1000.0 | Active | No PIN

>>> Test 4: Minimum Balance Enforcement

Creating Savings account with ₹300 (below minimum)

Error: Savings account needs minimum ₹500.0

Setting balance to minimum ₹500.0

Account #1004 | Bob Wilson (25 yrs) | Savings | ₹500.0 | Active | No PIN

>>> Test 5: Withdrawal with Minimum Balance

Initial: Account #1005 | Alice Brown (30 yrs) | Current | ₹1000.0 | Active
| No PIN

Withdrawn: ₹200.0

New balance: ₹800.0

After withdrawing ₹200: Account #1005 | Alice Brown (30 yrs) | Current | ₹800.0 | Active | No PIN

Trying to withdraw ₹900 (would leave ₹-100)

Error: Cannot withdraw. Minimum balance of ₹1000.0 required.

>>> Test 6: Account Status Management

Initial: Account #1006 | Charlie Green (35 yrs) | Savings | ₹2000.0 | Active | No PIN

Account #1006 has been closed.

After close: Account #1006 | Charlie Green (35 yrs) | Savings | ₹2000.0 | Inactive | No PIN

Trying to deposit to closed account:

Error: Account is inactive. Cannot deposit.

Account #1006 has been reopened.

After reopen: Account #1006 | Charlie Green (35 yrs) | Savings | ₹2000.0 | Active | No PIN

>>> Test 7: PIN Protection

PIN set successfully.

Withdraw with correct PIN (1234):

Withdrawn: ₹200.0

New balance: ₹1300.0

Withdraw with incorrect PIN (9999):

Error: Incorrect PIN.

Withdraw without providing PIN:

Error: PIN required for withdrawal

>>> Test 8: All Accounts Summary

Account #1001 | John Doe (25 yrs) | Savings | ₹1000.0 | Active | No PIN

Account #1002 | Young Kid (18 yrs) | Savings | ₹500.0 | Active | No PIN

Account #1003 | Test User (25 yrs) | Savings | ₹1000.0 | Active | No PIN

Account #1004 | Bob Wilson (25 yrs) | Savings | ₹500.0 | Active | No PIN

Account #1005 | Alice Brown (30 yrs) | Current | ₹800.0 | Active | No PIN

Account #1006 | Charlie Green (35 yrs) | Savings | ₹2000.0 | Active | No PIN

Account #1007 | Diana Prince (28 yrs) | Savings | ₹1300.0 | Active | PIN Set

=====

ENHANCED TEST COMPLETED!

=====

Key Python Features Used

1. **f-strings**: Clean string formatting with `f"Balance: ₹{self.balance}"`
2. **Private methods**: Methods starting with `_` (convention-based)
3. **Type hints**: Optional but helpful: `def deposit(self, amount: float) -> None:`
4. **str**: Python's version of `toString()`
5. **if name == "main"**: Standard Python entry point pattern